JDE MCP Incalus

OAuth 2.0 Authorization Code Flow + PKCE

A complete walkthrough of how OAuth authentication works in the JD Edwards MCP server — file by file, step by step, with data storage details at every stage.

Property	Value
Server	JD Edwards MCP Incalus (FastMCP + Starlette)
Port	8005
OAuth Flow	Authorization Code + PKCE (S256)
Client	claude-desktop (public client, no secret)
Login UI	http://localhost:5173/login
Auth Status	ACTIVE — enforced on all MCP tools
Token Storage	In-memory Python dicts (volatile)
Access Token TTL	1 hour
Refresh Token TTL	30 days (rotated on each use)
Session TTL	8 hours

Table of Contents

This document describes the currently active, running state of the OAuth implementation.

1. Architecture Overview

The system uses the **Authorization Code flow with PKCE** (Proof Key for Code Exchange), the recommended OAuth 2.0 flow for public clients that cannot securely store a client secret. Claude Desktop initiates the flow, the user authenticates via a React login UI, and the server issues short-lived access tokens (1 hour) backed by long-lived refresh tokens (30 days).

Components

Component	Role
Claude Desktop	The MCP client. Initiates OAuth, receives tokens, calls tools with Bearer auth.
Starlette App	HTTP server (uvicorn on port 8005). Routes: /oauth/login (POST), /oauth/callback (GET), and mounts FastMCP at /.
FastMCP	MCP framework. Configured with auth_server_provider and auth settings — validates access tokens on every tool call.
JDEOAuthProvider	The core OAuth logic implementing OAuthAuthorizationServerProvider. Handles authorize, token exchange, refresh, revoke.
5 In-Memory Stores	Python dicts with threading.Lock. Store OAuth requests, user sessions, auth codes, access tokens, refresh tokens.
JDE AIS Client	HTTP client that calls JD Edwards REST API to authenticate users and get AIS session tokens.
React Login UI	Frontend at localhost:5173 where the user enters JDE credentials.

2. File-by-File Breakdown

Every file in the `auth/` directory and its role in the OAuth flow.

2.1 server.py — Entry Point

Creates the **FastMCP** server instance with `auth_server_provider=provider` and `auth=auth_settings` — both **uncommented and active**. Also builds a **Starlette** application that serves three routes:

- `POST /oauth/login` → calls oauth_login handler
- `GET /oauth/callback` → calls oauth_callback handler
- `Mount /` → `mcp.streamable_http_app()` → all MCP tools

The app runs via `uvicorn.run(app, host='0.0.0.0', port=8005)`.

2.2 auth/config.py — Configuration Constants

Constant	Value	Purpose
CLIENT_ID	"claude-desktop"	The only registered client — no dynamic registration
REDIRECT_URI	"https://claude.ai/api/mcp/auth_callback"	Where Claude receives the auth code
LOGIN_URL	"http://localhost:5173/login"	React UI for user credentials
SCOPES	["openid", "profile", "mcp"]	Permission scopes; 'mcp' is required for tools
ACCESS_TOKEN_EXPIRY	timedelta(minutes=15)	Short-lived (overridden by provider to 1 hour)
REFRESH_TOKEN_EXPIRY	timedelta(hours=8)	Long-lived (overridden by provider to 30 days)
OAUTH_REQUEST_EXPIRY	timedelta(minutes=5)	How long the initial request object lives
SESSION_EXPIRY	timedelta(hours=8)	User session TTL

2.3 auth/models.py — Data Models

Eight dataclasses that define the shape of every object in the OAuth flow:

Class	Key Fields	Purpose
OAuthRequest	request_id, client_id, redirect_uri, state, code_challenge, timestamps	Saved when Claude initiates authorization
UserSession	session_id, username, ais_token, created_at, last_access, expires_at	Created after JDE AIS login succeeds
AuthenticationResult	session_id, username, environment, ais_token	Returned from JDE auth service to the route handler
JDEAuthorizationCode	code, session_id, client_id, redirect_uri, code_challenge, scopes, expires_at	The one-time authorization code
AuthorizationCodeEntry	authorization_code (object), session_id	Wrapper linking auth code to session
JDEAccessToken / AccessTokenEntry	token, session_id, username, scopes, expires_at	Short-lived access token with wrapper
JDERefreshToken / RefreshTokenEntry	token, session_id, expires_at	Long-lived refresh token with wrapper

2.4 auth/dependencies.py — Dependency Injection

Instantiates all five stores as module-level singletons, plus the JDE HTTP client and auth service. Wires everything into the `JDEOAuthProvider` constructor. These singletons persist for the lifetime of the server process.

```
oauth_request_store = OAuthRequestStore() session_store = SessionStore() authorization_code_store =
AuthorizationCodeStore() access_token_store = AccessTokenStore() refresh_token_store =
RefreshTokenStore() jde_client = JDEClient(base_url=os.environ.get("JDE_BASE_URL", "..."))
auth_service = JDEAuthService(jde_client=jde_client, session_store=session_store) provider =
JDEOAuthProvider( auth_service=auth_service, oauth_request_store=oauth_request_store,
authorization_code_store=authorization_code_store, session_store=session_store,
access_token_store=access_token_store, refresh_token_store=refresh_token_store, )
```

2.5 auth/jde_oauth_provider.py — OAuth Logic (444 lines)

The heart of the OAuth implementation. Extends `OAuthAuthorizationServerProvider` from the MCP library. Implements these methods:

Method	Purpose	Phase
get_client(client_id)	Validates client_id = 'claude-desktop'	Phase 1
authorize(client, params)	Creates OAuthRequest, returns LOGIN_URL?request_id=...	Phase 1
load_authorization_request(request_id)	Loads the OAuthRequest by ID	Phase 2
load_session(session_id)	Loads UserSession by ID	Phase 2
create_authorization_code(request_id, session_id)	Creates one-time code, deletes original request, returns redirect URL	Phase 3
load_authorization_code(client, code)	Validates an auth code belongs to the client	Phase 4
exchange_authorization_code(client, code)	Validates PKCE, exchanges code for tokens, deletes code	Phase 4
_issue_tokens(client, session_id, subject, scopes)	Internal: generates access + refresh tokens, stores both	Phase 4
load_access_token(token)	Validates an access token	Phase 5
load_refresh_token(client, token)	Validates a refresh token	Phase 6
exchange_refresh_token(client, token, scopes)	Rotates refresh token, issues new token pair	Phase 6
revoke_token(token)	Deletes the token from the appropriate store	Any time

2.6 auth/routes/oauth_routes.py — HTTP Endpoint Handlers

POST /oauth/login

Accepts JSON body: `{request_id, username, password, environment}`. Calls `auth_service.authenticate()` which: (1) POSTs to JDE AIS to get a token, (2) creates a UserSession in session_store. Then calls `provider.create_authorization_code(request_id, session.session_id)` to issue the one-time code. Returns JSON: `{'redirect_url': '...'}`.

GET /oauth/callback

Backend-to-backend callback. Accepts `?request_id=...&session;_id=...` query params. Calls `provider.create_authorization_code()` and redirects the browser to Claude's callback URL with the code and state.

2.7 auth/stores/ — In-Memory Data Stores (5 files)

Store File	Data	Key	TTL	Purpose
oauth_request_store.py	OAuthRequest objects	request_id (UUID hex)	5 min	Track PKCE challenge + state per authorization attempt
session_store.py	UserSession objects	session_id (UUID hex)	8 hours	Persist JDE AIS token after successful login
authorization_code_store.py	AuthorizationCodeEntry	code (random string)	10 min	Hold one-time code before exchange for tokens
access_token_store.py	AccessTokenEntry	token (random string)	1 hour	Verify every MCP tool call; extract session_id from claims
refresh_token_store.py	RefreshTokenEntry	token (random string)	30 days	Issue new access tokens without re-authentication; rotated on use

Each store uses `threading.Lock` for thread safety. Expired entries are checked on access (lazy eviction via `get()`), with optional `cleanup()` for bulk removal of stale records.

2.8 auth/services/ — Service Layer (2 files)

auth/services/jde_client.py

Makes a single HTTP POST to `{JDE_BASE_URL}/jderest/tokenrequest` with `{username, password, environment}`. Returns the JDE AIS response containing `userInfo.token`. Uses httpx with 30-second timeout.

auth/services/jde_auth_service.py

Orchestrates authentication: calls `jde_client.request_token()`, extracts the AIS token, creates a `UserSession` in the session store, and returns an `AuthenticationResult` with session_id, username, environment, and ais_token.

2.9 auth/oauth_client.py — Static Client Definition

Defines Claude Desktop as an `OAuthClientInformationFull` with: client_id='claude-desktop', redirect_uri='http://localhost:33418/oauth/callback', authorization_code and refresh_token grant types. Note: this client definition uses a different redirect_uri (localhost:33418) than the one in config.py (claude.ai).

3. Complete OAuth Flow — Step by Step

The flow is divided into 6 phases. Each phase lists every step, the files involved, and what data is stored at that point.

Phase 1: Claude Initiates Authorization

Step	What Happens	Files Involved	Data Stored
1	Claude wants to call an MCP tool. FastMCP's auth layer intercepts and requires OAuth. Claude sends an authorization request with client_id='claude-desktop', PKCE code_challenge (S256), and state.	MCP protocol → JDEOAuthProvider.get_client()	Nothing
2	get_client() validates the client_id against AuthConfig.CLIENT_ID. Returns the client configuration including allowed redirect_uris, grant_types, and scopes.	auth/config.py (line 14), auth/jde_oauth_provider.py (lines 54-77)	Nothing
3	FastMCP calls authorize(). An OAuthRequest is created: request_id (UUID), client_id, redirect_uri, state, PKCE code_challenge, created_at, expires_at (5 min).	JDEOAuthProvider.authorize() → OAuthRequestStore.create()	**OAuthRequestStore._store**: {request_id → OAuthRequest(PKCE challenge, state, redirect_uri)} — 5 min TTL
4	Returns the login URL: http://localhost:5173/login?request_id=	auth/config.py LOGIN_URL	—

Phase 2: User Logs In via React UI

Step	What Happens	Files Involved	Data Stored
5	Claude opens a browser to the login URL. The React UI loads and may call load_authorization_request(request_id) to get context (PKCE challenge, state, etc.)	React UI → JDEOAuthProvider.load_authorization_request()	Read from OAuthRequestStore
6	User enters JDE username, password, environment and clicks Login. React POSTs to http://localhost:8005/oauth/login with JSON body.	React UI → oauth_routes.py oauth_login()	—
7	oauth_login() calls auth_service.authenticate(username, password, environment)	oauth_routes.py (line 31) → JDEAuthService.authenticate()	—
8	authenticate() calls jde_client.request_token() which POSTs to JDE AIS API.	auth/services/jde_client.py (lines 12-34)	—
9	JDE AIS returns {userInfo: {token: '...'}, username: '...'}. The AIS token is extracted.	jde_client.py (line 28)	—
10	A UserSession is created: session_id (UUID), username, ais_token, created_at, last_access, expires_at (8 hrs).	JDEAuthService.authenticate() → SessionStore.create()	**SessionStore._store**: {session_id → UserSession(username, JDE AIS token)} — 8 hr TTL
11	authenticate() returns AuthenticationResult{session_id, username, environment, ais_token}.	jde_auth_service.py (lines 37-42)	—
12	Back in oauth_login(), calls provider.create_authorization_code(request_id, session.session_id).	oauth_routes.py (line 38) → JDEOAuthProvider.create_authorization_code()	—

Phase 3: Authorization Code Issued

Step	What Happens	Files Involved	Data Stored
13	create_authorization_code() loads the original OAuthRequest (gets PKCE, state, redirect_uri) and the UserSession (gets username, AIS token).	jde_oauth_provider.py lines 212-225	Read from OAuthRequestStore + SessionStore
14	Generates a random one-time authorization code (32 bytes, url-safe base64). Creates an AuthorizationCode with all required fields.	jde_oauth_provider.py lines 228-239	—
15	Saves the authorization code in AuthorizationCodeStore, linked to the session_id.	jde_oauth_provider.py lines 241-246 → AuthorizationCodeStore.save()	**AuthorizationCodeStore._store**: {code → AuthorizationCodeEntry(code, session_id)} — 10 min TTL
16	Deletes the original OAuthRequest from OAuthRequestStore (one-time use).	jde_oauth_provider.py line 248 → OAuthRequestStore.delete()	OAuthRequestStore: entry removed
17	Returns redirect URL: redirect_uri?code=&state;=. The redirect_uri comes from the original OAuthRequest, which was provided by Claude.	jde_oauth_provider.py lines 250-257	—
18	oauth_login() returns {'redirect_url': '...'} as JSON to the React UI.	oauth_routes.py lines 43-46	—
19	React UI redirects the browser to the redirect URL, which goes to Claude's callback.	Browser redirect → Claude Desktop	—

Phase 4: Claude Exchanges Code for Tokens

Step	What Happens	Files Involved	Data Stored
20	Claude receives the auth code at its callback URL. It calls the FastMCP token endpoint with grant_type=authorization_code, code, and code_verifier.	MCP protocol → JDEOAuthProvider	—
21	FastMCP calls load_authorization_code(client, code). Loads the AuthorizationCodeEntry, validates it exists and belongs to this client (client_id match).	jde_oauth_provider.py lines 188-204 → AuthorizationCodeStore.get()	Read from AuthorizationCodeStore
22	Optionally, verify_pkce() is called: hashes code_verifier with SHA-256, base64url-encodes, compares to stored code_challenge. Uses secrets.compare_digest() for constant-time comparison.	jde_oauth_provider.py lines 79-105	—
23	exchange_authorization_code() loads the code entry and the linked UserSession, then deletes the auth code (one-time use, cannot be replayed).	jde_oauth_provider.py lines 315-356	AuthorizationCodeStore: entry removed
24	_issue_tokens() generates two random tokens (48 bytes each): access_token (1 hour TTL) and refresh_token (30 day TTL). Access token includes claims: {jde: {session_id: '...'}} for session lookup.	jde_oauth_provider.py lines 259-313	—
25	AccessTokenEntry saved: keyed by access_token string, stores the full access token object and session_id.	jde_oauth_provider.py lines 294-299 → AccessTokenStore.save()	**AccessTokenStore._store**: {token → AccessTokenEntry(access_token, session_id)} — 1 hr TTL
26	RefreshTokenEntry saved: keyed by refresh_token string, stores the full refresh token object and session_id.	jde_oauth_provider.py lines 301-306 → RefreshTokenStore.save()	**RefreshTokenStore._store**: {token → RefreshTokenEntry(refresh_token, session_id)} — 30 day TTL

Step	What Happens	Files Involved	Data Stored
27	Returns OAuthToken to Claude: {access_token, refresh_token, expires_in=3600, scope='openid profile mcp'}.	jde_oauth_provider.py lines 308-313	—

Phase 5: Claude Uses MCP Tools

Step	What Happens	Files Involved	Data Stored
28	Claude calls any MCP tool (e.g., jde_run_sql_query, jde_customer_ledger_inquiry) with Authorization: Bearer header.	MCP protocol → FastMCP	—
29	FastMCP calls load_access_token(token) to verify the token exists and hasn't expired. If expired, the entry is deleted from AccessTokenStore and None is returned.	JDEOAuthProvider.load_access_token() → AccessTokenStore.get()	Read from AccessTokenStore (expired entries auto-deleted)
30	FastMCP checks the token has the required 'mcp' scope (from required_scopes=['mcp'] in AuthSettings at server.py lines 66-70). If valid, the tool executes. If invalid, returns 401.	server.py lines 66-70 + FastMCP auth layer	—

Phase 6: Token Refresh

Step	What Happens	Files Involved	Data Stored
31	When the access token expires (1 hour), Claude gets a 401. It sends a refresh request: grant_type=refresh_token, refresh_token=.	MCP protocol → FastMCP	—
32	FastMCP calls load_refresh_token(client, token). Validates existence, expiry, and client_id match.	jde_oauth_provider.py lines 371-385 → RefreshTokenStore.get()	Read from RefreshTokenStore
33	exchange_refresh_token() loads the linked UserSession from SessionStore, then deletes the OLD refresh token (rotation — cannot be reused).	jde_oauth_provider.py lines 410-444	RefreshTokenStore: old token deleted
34	Calls _issue_tokens() to generate a completely new access token + refresh token pair. Both are stored; the new refresh token has a fresh 30-day TTL.	jde_oauth_provider.py lines 259-313	**AccessTokenStore:** new token stored **RefreshTokenStore:** new token stored

4. Data Storage Summary

All data is stored **in-memory** (Python dicts with threading.Lock). There is no database, no Redis, no file-based persistence. If the Python process restarts, all sessions, authorization codes, and tokens are lost.

Complete State at Each Stage

Stage	Store(s)	What's Stored	TTL
After authorize()	OAuthRequestStore	{request_id → OAuthRequest(PKCE challenge, state, redirect_uri)}	5 min
After oauth_login()	SessionStore	{session_id → UserSession(username, JDE AIS token)}	8 hours
After create_authorization_code()	OAuthRequestStore + AuthorizationCodeStore	Request deleted. {code → AuthorizationCodeEntry(code, session_id)} stored.	10 min
After exchange_authorization_code()	AccessTokenStore + RefreshTokenStore	Auth code deleted. {token → AccessTokenEntry} + {token → RefreshTokenEntry} stored.	1 hr / 30 days
After exchange_refresh_token()	AccessTokenStore + RefreshTokenStore	Old refresh token deleted. New token pair stored.	1 hr / 30 days
Each MCP tool call	AccessTokenStore	Token looked up and validated (lazy expiry check)	—
Token revocation	AccessTokenStore or RefreshTokenStore	Token deleted from appropriate store	—

Key: Token Rotation on Refresh

When a refresh token is used (`exchange_refresh_token()`), the OLD refresh token is **deleted** from RefreshTokenStore and a completely NEW token pair (access + refresh) is issued. This means a compromised refresh token can only be used once — after rotation, the old one is worthless. This is a security best practice.

TTL Values Used by the Provider (Actual Running Values)

Token Type	Actual TTL	Source
Access Token	1 hour (3600 seconds)	Defined as ACCESS_TOKEN_TTL = 3600 in jde_oauth_provider.py line 29
Refresh Token	30 days (30 × 86400 seconds)	Defined as REFRESH_TOKEN_TTL = 30 * 86400 in jde_oauth_provider.py line 30
Auth Code	10 minutes	AUTH_CODE_LIFETIME = timedelta(minutes=10) in jde_oauth_provider.py line 34

Note: AuthConfig defines different values (access=15min, refresh=8hr) but the provider overrides these with the hardcoded values above.

5. Key Observations

Auth is Now Fully Active

The `auth_server_provider=provider` and `auth=auth_settings` parameters in `server.py` (lines 78-81) are **uncommented**. FastMCP enforces OAuth token validation on every MCP tool call. The Starlette app (lines 870-892) is also uncommented and runs via uvicorn.

No Persistence — Fully Volatile

All five stores are plain Python `dicts`. If the server process restarts: all user sessions, authorization codes, access tokens, and refresh tokens are **permanently lost**. Users would need to re-authenticate. For production, these stores should be backed by Redis or a database.

No JWT — Tokens Are Random Strings

Tokens are generated via `secrets.token_urlsafe(48)` — random 48-byte base64url-encoded strings. They are **not JWT-encoded**. The `auth/services/jwt_service.py` file is empty. This means token validation requires a lookup in the in-memory store, which doesn't scale horizontally (no shared state across server instances).

Token Rotation on Refresh

The refresh token is **rotated on each use**: the old refresh token is deleted and a completely new pair (access + refresh) is issued. This prevents replay attacks with stolen refresh tokens.

PKCE with S256

PKCE uses SHA-256 hashing (method 'S256'). The code_verifier (secret) is never stored on the server — only the code_challenge (hash) is stored in the OAuthRequest. Verification uses `secrets.compare_digest()` for constant-time string comparison, preventing timing attacks.

Two Different Redirect URIs

`auth/config.py` (line 16) defines `REDIRECT_URI = 'https://claude.ai/api/mcp/auth_callback'`. `auth/oauth_client.py` (line 8) uses `'http://localhost:33418/oauth/callback'`. These serve different purposes: config.py is for the provider's client registration, oauth_client.py is an alternative static client definition that is not currently wired.

Thread Safety via Locking

Every store uses `threading.Lock` (via `with self._lock`) on all read and write operations. This ensures that concurrent requests don't corrupt the in-memory state. However, this also means the stores are a potential bottleneck under high concurrency.

Appendix: Stale Code Notices

auth/routes/oauth_routes.py — Commented Line

Line 36 contains `# session_id = session_store.create(session)` which is commented out. This is correct behavior — the session is already created inside `auth_service.authenticate()` (in jde_auth_service.py line 32). The commented line is stale and should be removed for clarity.

auth/oauth_client.py — Standalone Definition

This file defines a separate `OAuthClientInformationFull` for Claude Desktop but it is **not imported or used** anywhere in the current codebase. The client definition used at runtime comes from `JDEOAuthProvider.get_client()` which reads from `AuthConfig`. This file may be a leftover from an earlier approach.

auth/services/jwt_service.py — Empty

This file exists but has no content. Token validation is done by looking up the token string in the in-memory stores rather than by verifying a JWT signature. If JWT-encoded tokens are desired in the future, this file would contain the signing and verification logic.